Министерство образования и исследований
Государственный университет Молдовы
Отдел математики и информатики

ОТЧЕТ
по лабораторной работе №4
«Продвинутые объекты в JavaScript»

Выполнил: студент гр. IA2503ru
Б. Антонович

Проверил: Н. Калин

Кишинёв 2026

1. Теоретическая часть

1.1. Формулировка задачи

Цель лабораторной работы состоит в создании консольного приложения на JavaScript для работы с предметами инвентаря. Программа хранит данные об объектах, меняет их свойства и выполняет основные действия с ними.

В работе нужно создать класс Item для обычных предметов и класс Weapon для оружия. Класс Weapon наследует Item. Также нужно сделать такую же логику через функции-конструкторы. Дополнительно требуется использовать опциональную цепочку при обращении к свойствам объекта.

Каждый предмет хранит название, вес и редкость. Оружие хранит еще урон и прочность. Для предметов нужно реализовать вывод информации, изменение веса, использование оружия и его ремонт.

1.2. Описание цели и основные этапы работы

Цель работы состоит в изучении классов, наследования, методов объектов и функций-конструкторов в JavaScript. Эти возможности применяются на простой модели инвентаря.

Основные этапы работы такие. Сначала создаются классы Item и Weapon. Затем пишется тестирование объектов и проверка методов. После этого добавляется опциональная цепочка. В конце классы переписываются через функции-конструкторы с тем же набором свойств и методов.

2. Практическая часть

2.1. Краткое описание особенностей реализации

Работа выполнена в одном файле JavaScript. В коде есть две версии одной логики. Первая версия построена на классах. Вторая версия построена на функциях-конструкторах. Такой подход помогает увидеть разницу между двумя способами создания объектов.

В классе Item хранятся name, weight и rarity. Метод getInfo() возвращает строку с данными предмета.

```
class Item {
  /** @type {string} */ name;
  /** @type {number} */ weight;
  /** @type {string} */ rarity;

  constructor(name, weight, rarity) {
    const validRarities = ["common", "uncommon", "rare", "legendary"];
    if (typeof weight !== "number" || weight <= 0) {
      console.log("Ошибка: вес должен быть положительным числом.");
      return;
    }
    if (!validRarities.includes(rarity)) {
      console.log("Ошибка: недопустимое значение редкости.");
      return;
    }
    this.name = name;
    this.weight = weight;
    this.rarity = rarity;
  }

  /**
   * Возвращает строку с информацией о предмете.
   * @returns {string} Информация о предмете.
   */
  getInfo() {
    return `Name: ${this.name}, Weight: ${this.weight}, Rarity: ${this.rarity}`;
  }

  /**
   * Изменяет вес предмета.
   * @param {number} newWeight - Новый вес (положительное число).
   * @returns {void}
   */
  setWeight(newWeight) {
    if (typeof newWeight !== "number" || newWeight <= 0) {
      console.log("Ошибка: вес должен быть положительным числом.");
      return;
    }
    this.weight = newWeight;
  }
}
```

Метод setWeight() меняет вес. В классе Weapon добавлены damage и durability. Метод use() уменьшает прочность на 10, а repair() возвращает прочность к 100.

```
class Weapon extends Item {
  /** @type {number} */ damage;
  /** @type {number} */ durability;

  constructor(name, weight, rarity, damage, durability) {
    super(name, weight, rarity); // передаём поля родителю
    if (typeof damage !== "number" || damage <= 0) {
      console.log("Ошибка: урон должен быть положительным числом.");
      return;
    }
    if (typeof durability !== "number" || durability < 0 || durability > 100) {
      console.log("Ошибка: прочность должна быть числом от 0 до 100.");
      return;
    }
    this.damage = damage;
    this.durability = durability;
  }

  /**
   * Уменьшает прочность на 10 при использовании. Не уходит ниже 0.
   * @returns {void}
   */
  use() {
    if (this.durability > 0) {
      this.durability = Math.max(0, this.durability - 10);
    }
  }

  /**
   * Восстанавливает прочность до 100.
   * @returns {void}
   */
  repair() {
    this.durability = 100;
  }
}
```

Для большей надежности в код добавлена проверка входных данных. Вес, урон и прочность должны быть числами. Редкость должна совпадать с одним из допустимых значений. Это помогает избежать ошибок при создании объектов.

Опциональная цепочка используется при обращении к объекту, который может быть null или undefined. В таком случае программа не падает с ошибкой и возвращает undefined.

Версия на функциях-конструкторах повторяет ту же структуру. Свойства создаются через this, а методы добавляются через prototype. Наследование для оружия настроено через call() и Object.create().

```
// настраиваем наследование прототипа
WeaponConstructor.prototype = Object.create(ItemConstructor.prototype);
WeaponConstructor.prototype.constructor = WeaponConstructor;
```

В конце файла идет тестирование. Создаются несколько объектов Item и Weapon. После этого вызываются методы getInfo(), setWeight(), use() и repair(). Это показывает, что код работает в обычных сценариях.

```
  /**
   * Возвращает строку с информацией о предмете.
   * @returns {string} Информация о предмете.
   */
  getInfo() {
    return `Name: ${this.name}, Weight: ${this.weight}, Rarity: ${this.rarity}`;
  }

  /**
   * Изменяет вес предмета.
   * @param {number} newWeight - Новый вес (положительное число).
   * @returns {void}
   */
  setWeight(newWeight) {
    if (typeof newWeight !== "number" || newWeight <= 0) {
      console.log("Ошибка: вес должен быть положительным числом.");
      return;
    }
    this.weight = newWeight;
  }
}
```

```
  /**
   * Уменьшает прочность на 10 при использовании. Не уходит ниже 0.
   * @returns {void}
   */
  use() {
    if (this.durability > 0) {
      this.durability = Math.max(0, this.durability - 10);
    }
  }

  /**
   * Восстанавливает прочность до 100.
   * @returns {void}
   */
  repair() {
    this.durability = 100;
  }
}
```

3. Вывод

В ходе лабораторной работы было создано консольное приложение на JavaScript для работы с предметами инвентаря. Были изучены классы, наследование, методы объектов и функции-конструкторы.

Также была освоена опциональная цепочка. Она помогает безопасно обращаться к объектам, которые могут отсутствовать. Это делает код удобнее и надежнее.

Ссылка на репозиторий GitHub: https://github.com/badm1nton1/jslabs/tree/main/lab4

4. Ответы на контрольные вопросы

1. Какое значение имеет this в методах класса?

В методах класса this указывает на текущий объект. Через this метод получает доступ к свойствам и другим методам этого объекта. Например, в getInfo() через this читаются name, weight и rarity.

2. Как работает модификатор доступа # в JavaScript?

Знак # перед именем поля делает его приватным. Такое поле видно только внутри класса. Снаружи объекта обратиться к нему нельзя. Это помогает скрывать внутренние данные и защищать их от прямого изменения.

3. В чем разница между классами и функциями-конструкторами?

Классы дают более короткий и понятный синтаксис для создания объектов и наследования. Функции-конструкторы делают то же самое, но через старый подход с this, prototype и call(). По сути, они решают одну задачу, но записываются по-разному.

5. Список использованных источников

1. MDN Web Docs. Array.
URL: https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/Array
2. MDN Web Docs. Set.
URL: https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/Set
3. MDN Web Docs. Optional chaining.
URL: https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Operators/Optional_chaining
4. JSDoc. Документирование JavaScript кода.
URL: https://jsdoc.app/
5. Задание к лабораторной работе №4, кафедра FAF, ТУМ, 2026.